



Universidade Estadual de Campinas – Unicamp
Faculdade de Engenharia Elétrica e de
Computação – FEEC



IA367 - Introdução ao Aprendizado por Reforço

Turma - H

Lista de Exercícios 3

Alunos

João Victor Gomes Carneiro RA 239738

Gabriel Buzzi Sanches RA 256389

2) Algoritmo PPO para o Cart Pole

A estrutura do agente consiste em duas redes MLP para o actor-critic contendo 64 neurônios por camada cada, função de ativação `tanh` e saídas específicas para cada uma: a rede do actor produz *logits* para uma política categórica sobre as duas ações disponíveis, enquanto a rede do crítico estima o valor da recompensa dada uma observação. A interação com o ambiente segue o padrão on-policy: em cada época são coletados 4.000 passos, a partir dos quais calculamos vantagens via Generalized Advantage Estimation (GAE) e os returns-to-go e então realizamos atualizações de política e de valor sobre esse mesmo lote de dados.

Os hiperparâmetros foram escolhidos para equilibrar estabilidade e velocidade de convergência. O desconto temporal $\gamma=0,99$ define um horizonte efetivo relativamente longo, apropriado para a dinâmica do *CartPole*. O parâmetro $\lambda=0,97$, utilizado no GAE, reduz a variância das estimativas de vantagem sem introduzir viés excessivo. A atualização de política segue o objetivo PPO-Clip com razão de corte (*clip ratio*) de 0,2, isso limita a mudança da política entre épocas, prevenindo passos muito agressivos que degradariam o desempenho. Para otimização, emprega-se Adam com taxa de aprendizado de 3×10^{-4} para o actor e 10^{-3} para o crítico. Em cada época são realizadas até 80 iterações de atualização tanto para a rede de ação como para a rede do crítico usando o mesmo conjunto de trajetórias; o uso de um alvo de divergência KL de 0,01 implementa uma parada antecipada caso a nova política se afaste demais da política comportamental, reforçando a estabilidade do treino. No total o treino é realizado em 30 épocas.

O ciclo de aprendizado ocorre da seguinte maneira. Durante a coleta, a política corrente amostra ações a partir dos *logits* do ator; as transições (estado, ação, recompensa, log-probabilidade antiga e valor) são armazenadas em um buffer. Ao final de um episódio ou do lote, computamos as vantagens por meio das diferenças temporais ($\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$) suavizadas pelo fator $\gamma\lambda$ e calculamos os returns-to-go para supervisionar o crítico. Na etapa de otimização, o ator é ajustado maximizando o objetivo clipado do PPO, e o crítico é ajustado por regressão quadrática para aproximar os returns-to-go. O monitoramento contínuo da divergência KL aproxima a mudança efetiva de política; caso o KL supere $1,5 \times$ o alvo, interrompemos as iterações de política naquela época para preservar a proximidade entre política nova e antiga.

Resultados obtidos do treino:

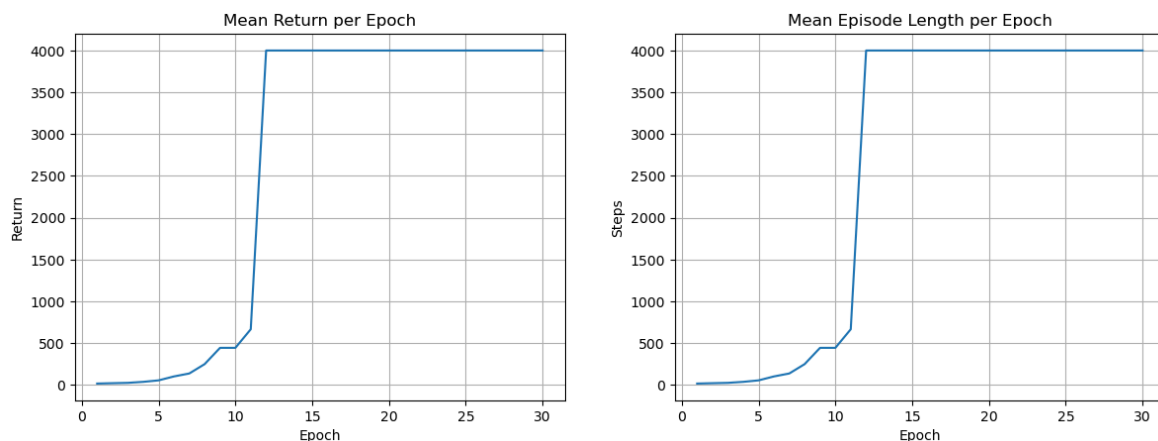


Figura 1: Média do retorno e média do tamanho da época ao longo do treinamento.

Quanto à convergência, o CartPole-v1 é tradicionalmente considerado resolvido quando o retorno médio se aproxima do teto do ambiente (cerca de 500 pontos por episódio, limitado pelo tempo). Na prática, observamos uma curva de aprendizado em que o retorno médio por época cresce rapidamente nas primeiras iterações e, à medida que o crítico melhora suas estimativas e o ator estabiliza as probabilidades de ação, o desempenho se mantém próximo do máximo. O salto para 4000 indica um episódio cortado pelo fim da época e não pelo fim verdadeiro do episódio.

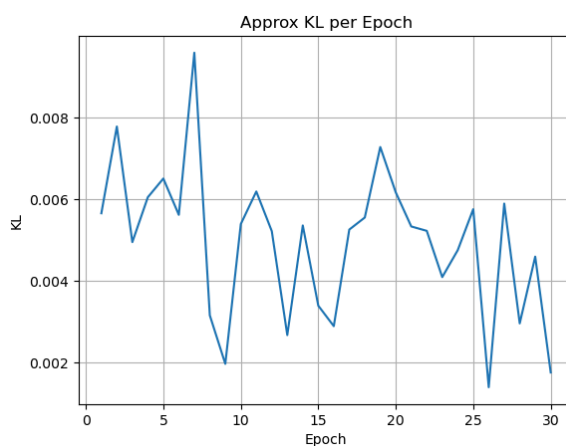


Figura 2: Valor de KL ao longo das épocas de treinamento.

As atualizações de política ficaram conservadoras, o KL médio ficou sempre abaixo do valor definido pelo `target_kl = 0.01` o que indica passos controlados e pouca necessidade de early stopping durante o treinamento.

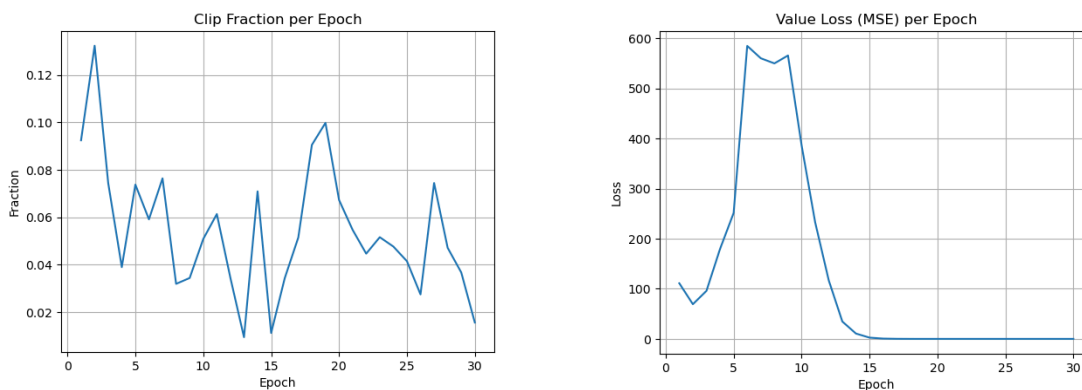


Figura 3: Fração do clip e Loss de valor ao longo das épocas de treinamento.

Por fim a “fração do clip” ficou baixa/moderada (~2%-13%) indicando que o clipping atuou quando preciso, sem “estrangular” o aprendizado. Bom equilíbrio entre estabilidade e avanço. O crítico aprende com MSE alta no começo, queda rápida a partir da 10ª época e estabilização próxima de zero.

Em síntese, a configuração adotada conduz a uma convergência rápida e estável no CartPole-v1. O uso de GAE, do objetivo clipado e do monitoramento de KL sustenta atualizações confiáveis da política, enquanto o crítico, treinado por MSE, fornece um sinal de valor consistente para orientar o ator.

5) O Algoritmo Genético (GA) para o Grid-World

a)

Para cada indivíduo é mapeada 12 ações aleatórias dentro das possíveis 4 opções, "UP", "DOWN", "LEFT", "RIGHT", para o agente seguir em cada posição do tabuleiro. Então essa política é avaliada e tem seu valor retornado:

Python

```
Individuo 1: ['UP' 'UP' 'RIGHT' 'LEFT' 'DOWN' 'DOWN' 'LEFT'
'DOWN' 'DOWN' 'UP' 'LEFT'
'LEFT']
Política: {(1, 1): np.str_('UP'), (1, 2): np.str_('UP'), (1, 3):
np.str_('RIGHT'), (2, 1): np.str_('LEFT'), (2, 2):
np.str_('DOWN'), (2, 3): np.str_('DOWN'), (3, 1):
np.str_('LEFT'), (3, 2): np.str_('DOWN'), (3, 3):
np.str_('DOWN'), (4, 1): np.str_('UP'), (4, 2): np.str_('LEFT'),
(4, 3): np.str_('LEFT')}
Fitness: -inf
```

```
Individuo 2: ['UP' 'UP' 'UP' 'UP' 'RIGHT' 'DOWN' 'RIGHT' 'UP'
'DOWN' 'UP' 'UP' 'DOWN']
Política: {(1, 1): np.str_('UP'), (1, 2): np.str_('UP'), (1, 3):
np.str_('UP'), (2, 1): np.str_('UP'), (2, 2): np.str_('RIGHT'),
(2, 3): np.str_('DOWN'), (3, 1): np.str_('RIGHT'), (3, 2):
np.str_('UP'), (3, 3): np.str_('DOWN'), (4, 1): np.str_('UP'),
(4, 2): np.str_('UP'), (4, 3): np.str_('DOWN')}
Fitness: -inf
```

```
Individuo 3: ['DOWN' 'UP' 'UP' 'RIGHT' 'UP' 'RIGHT' 'DOWN' 'UP'
'UP' 'UP' 'LEFT' 'DOWN']
Política: {(1, 1): np.str_('DOWN'), (1, 2): np.str_('UP'), (1,
3): np.str_('UP'), (2, 1): np.str_('RIGHT'), (2, 2):
np.str_('UP'), (2, 3): np.str_('RIGHT'), (3, 1): np.str_('DOWN'),
```



```
(3, 2): np.str_('UP'), (3, 3): np.str_('UP'), (4, 1):  
np.str_('UP'), (4, 2): np.str_('LEFT'), (4, 3): np.str_('DOWN')}  
Fitness: -0.5677309913233721
```

b)

Para cada filho, escolhem-se dois pais como os vencedores de dois torneios de tamanho 2 (baseados no fitness). Em seguida, faz-se crossover de 1 ponto: sorteia-se um índice `idx` e o cromossomo do filho é formado por `pai1[:idx]` seguido de `pai2[idx:]`

Python

```
def cruza(p1, p2):  
    cromossomo1 = p1.cromossomo  
    cromossomo2 = p2.cromossomo  
    idx = np.random.choice(len(cromossomo1))  
    return Indivíduo(np.append(cromossomo1[:idx], cromossomo2[idx:]))
```

c)

A mutação consiste basicamente na troca aleatória de uma ação possível, dentro das ações do indivíduo dentro da população. Então um novo indivíduo é retornado:

Python

```
def muta(p):  
    cromossomo = p.cromossomo  
    idx = np.random.choice(len(cromossomo))  
    cromossomo[idx] = np.random.choice(["UP", "DOWN", "LEFT", "RIGHT"])  
    return Indivíduo(cromossomo)
```

d)

O processo de seleção é dado através da escolha de 2 indivíduos com maiores valores de política através de dois torneios independentes dentre os possíveis indivíduos da população naquela iteração.

e)

Sim se o agente seguir a política obtida, $\pi = 0.3973$, partindo de (1,1) chegaria no seu objetivo positivo (4,3) de acordo com as ações indicadas pela política na tabela logo abaixo.

→	→	→	↑
↑	X	↑	↓
↑	→	←	←

f)

Aumentando para 200 o número de indivíduos na população e mantendo-se o número de gerações, obtém-se a seguinte política com $\pi = 0.4019$:

→	→	→	←
↑	X	←	←
↑	→	→	←

Aumentando o número de iterações (gerações) para 200 e mantendo-se o número de indivíduos em 100, obtém-se a seguinte política com $\pi = 0.3991$:

→	→	→	↓
↑	X	↑	↑
↑	←	↑	←

Apesar da diferença no valor de π ser pequena entre dobrar a quantidade de indivíduos e dobrar a quantidade de gerações, dobrar a quantidade de indivíduos para este problema parecer resultar em um maior valor de π .